# Coding Project 0: Hip Shop

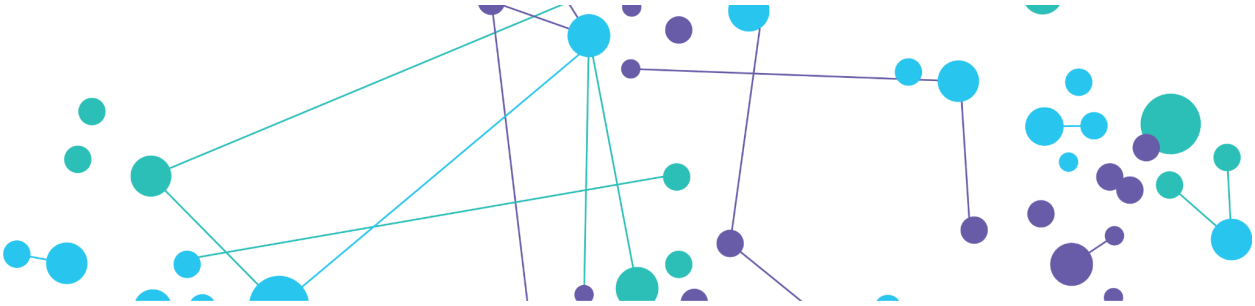
## Due: 9/18/26 at 11:59 PM

*Goals: Familiarity with Python, Gradescope, and the structure of coding assignments*
*This introductory assignment is adapted from UC Berkeley's Project 0.*

If clarifications are needed, they will be documented on this page.

If you are joining the course later, or had to be manually added to the course, we'll have to resync Gradescope and the Canvas Roster. Please post on Piazza if you don't have access to the Gradescope!

---



# Introduction

*Here's normally where we'd introduce the assignment and what you'll be doing.*

In this assignment, you'll be getting familiarized with Python and submitting your code to gradescope. You'll do so by modifying a few functions present in the starter code directory.

---

# Logistics

*Here's where I put clarifications on how to submit your assignment and grading breakdown.*

## Starter Code

*Here I'll give a layout of what your download structure looks like.*

```
coding_0/
├─ COLLABORATION.MD
├─ tests/
│   ├─ q1_test_addition.py
│   ├─ etc...
├─ test.py
├─ addition.py
├─ buyLotsOfFruit.py
├─ shop.py
├─ shopSmart.py
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

*Note: Collaboration.MD, tests/, and test.py will be present in most coding assignments.*

COLLABORATION.MD:

- An empty file where you will put your collaboration statement, listing anyone or anywhere you got help from. If you truly did not get help from anywhere or anyone, you must write that in your collaboration statement. **You will lose points if this is not completed.**

tests/:

- Most coding projects will have a tests folder that has instructor provided test cases for your code. You should be running these every time you implement new features. These are run using the test.py function we provide.

test.py:

- Most coding projects will have a test.py file, which provides an interface for running the tests in tests. You'll be learning how to use this in the Programming section.

addition.py:

- A small stub function that you'll modify to add two numbers, which will showcase that you know how basic syntax and functions in python works.

buyLotsOfFruit.py:

- A small stub function you'll modify to calculate the total cost of buying some fruit, given each fruit has a price per pound. This will test that you know how to use dictionaries and tuples in Python.

shop.py:

- This file provides a class representing a FruitShop, which is used in shopSmart.py

shopSmart.py:

- A small stub function you'll modify to find the most cost efficient shop to buy fruit at. This will test that you know how to use functions in different classes and manipulate objects in Python.

## Grade Breakdown

*Here is a breakdown of where your points will be coming from. Some of these components will be autograded. Some of the autograded test cases are visible to you, while others are hidden. Other components will be manually graded by the course staff. All coding projects will have a readability component to make sure your code is clean, concise, and readable.*

| **Feature** | **Points** | **Graded By** |
|---|---|---|
| Problem 1: Sum | 30<br>*5 tests, each worth 6 points* | Autograder |
| Problem 2: Buy Lots Of Fruit | 30<br>*5 tests, each worth 6 points* | Autograder |
| Problem 3: Shop Smart | 30<br>*5 tests, each worth 6 points* | Autograder |
| Style and Readability | 5 | Manual |
| Collaboration Statement | 5 | Manual |
| **Total** | 100 | N/A |

## Submission Instructions

*In this section will be any special instructions for submitting assignments to gradescope. However, for this (and most assignments) it'll look like this:*

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

In gradescope please upload **ONLY** the following files:

- addition.py
- buyLotsOfFruit.py
- shop.py
- shopSmart.py
- Collaboration.MD

---

# Background

*In this section may be some more information about the assignment that will help you succeed, it will look different for each assignment.*

## Install Python

If you don't already have python installed on your computer, follow these instructions.

- Get Python Set Up

You're free to use any IDE that you wish, but if you are looking for recommendations, Visual Studio Code is a popular choice.

**We will offer support at office hours for installing or maintaining Python on your laptop until this assignment is due. However, it is ultimately your responsibility to acquire and maintain the packages and environments you need when they arise.**

## Task: Familiarize yourself with Python

If you're new to Python or need a refresher, we would recommend the following materials.

- Berkeley's Python Basics
- (If you know any short materials that have helped you, please post on Piazza!)

This course assumes familiarity with programming basics, and neither the TA or Instructors will be expected to teach you programming (this excludes having trouble with tests or the autograder, which this assignment is attempting to give you practice with now).

## Task: Testing the Autograder

Right now we'll get some basics for getting familiar with Python and the autograder.

Open addition.py and look at the definition of add. You'll see an unfinished method stub:

```
def add(a, b):
    "Return the sum of a and b"
    "*** YOUR CODE HERE ***"
    return 0
```

**Let's run the autograder to test this specific problem.**

```
python test.py -q q1
```

You should see an output that looks like the following, showing that we're failing our tests for question 1:

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

```
Running tests for: q1

============================================================
Running TestAddition Tests
============================================================
✗ test_add_1: FAILED
✗ test_add_2: FAILED
✗ test_add_3: FAILED

----------------------------------------
TestAddition Results:
  Passed: 0/3
  Failed: 3/3
  Errors: 0/3
  ⚠️  Some TestAddition tests FAILED
----------------------------------------

============================================================
OVERALL TEST SUMMARY
============================================================
Total Tests Run: 3
Passed: 0
Failed: 3
Errors: 0

❌ 3 test(s) failed
============================================================
======================================================================
FAIL: test_add_1 (tests.q1_test_addition.TestAddition)
Check that 2+2 is 4
----------------------------------------------------------------------
Traceback (most recent call last):
  File "/home/your_directory/tests/q1_test_addition.py", line 14, in
test_add_1
    self.assertEqual(result, 4, f"Expected 4, got {result}")
AssertionError: 0 != 4 : Expected 4, got 0

======================================================================
FAIL: test_add_2 (tests.q1_test_addition.TestAddition)
Check that 2+6 is 8
----------------------------------------------------------------------
Traceback (most recent call last):
  File "/home/your_directory/q1_test_addition.py", line 19, in test_add_2
    self.assertEqual(result, 8, f"Expected 8, got {result}")
AssertionError: 0 != 8 : Expected 8, got 0

======================================================================
FAIL: test_add_3 (tests.q1_test_addition.TestAddition)
Check that 3+3 is 6
----------------------------------------------------------------------
Traceback (most recent call last):
  File "/home/your_directory/q1_test_addition.py", line 24, in test_add_3
    self.assertEqual(result, 6, f"Expected 6, got {result}")
AssertionError: 0 != 6 : Expected 6, got 0

----------------------------------------------------------------------
Ran 3 tests in 0.001s

FAILED (failures=3)
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

# Programming

*Note: While it may take some time to get a hang of Python's syntax, once you get the hang of it, we'd expect you to feel comfortable solving these problems. If you are struggling with solving these problems (even in your language of choice before Python), please see the instructor so we can discuss further about if this course is for you right now.*

## Problem 1: Sum [20 Points]

Modify addition.py to read:

```
def add(a, b):
    "Return the sum of a and b"
    print("Passed a = %s and b = %s, returning a + b = %s" % (a, b, a + b))
    return a + b
```

Test your implementation using:

```
python test.py -q q1
```

You now pass all tests. Notice the new lines "Passed a=…" which appear before your tests pass. These are produced by the print statement in add. You can use print statements like that to output information useful for debugging.

## Problem 2: Buy Lots of Fruit [30 Points]

Implement the buyLotsOfFruit(orderList) function in buyLotsOfFruit.py which takes a list of (fruit,numPounds) tuples and returns the cost of your list. If there is some fruit in the list which doesn't appear in fruitPrices return None. Please do not change the fruitPrices variable.

Run the tester until question 2 passes all tests.

```
python test.py -q q2
```

Each test will confirm that buyLotsOfFruit(orderList) returns the correct answer given various possible inputs.

## Problem 3: Shop Smart [30 Points]

Fill in the function shopSmart(orderList,fruitShops) in shopSmart.py, which takes an orderList (like the kind passed in to FruitShop.getPriceOfOrder) and a list of FruitShop and returns the FruitShop where your order costs the least amount in total.

Don't change the file name or variable names, please.

Run the tester until question 3 passes all tests.

Each test will confirm that shopSmart(orderList,fruitShops) returns the correct answer given various possible inputs.

---

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

# Wrapping Up and Submission

## Final Testing

Before submitting, test that all of your tests are passing by running the autograder with no optional parameters.

```
python test.py
```

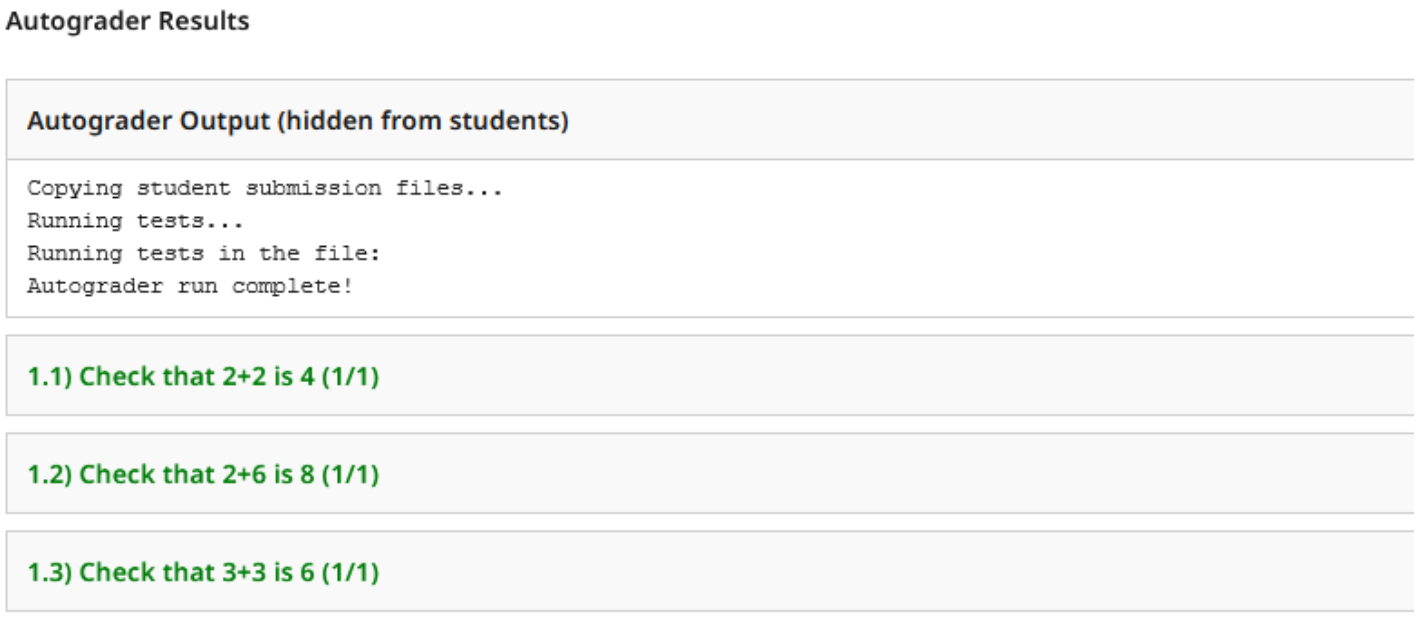
You should see that all of your tests are passing at this point.

## Submitting to Gradescope

In gradescope, go to Coding Project 0 and upload **ONLY** the following files:

- addition.py
- buyLotsOfFruit.py
- shop.py
- shopSmart.py
- Collaboration.MD

You should see that all your tests pass on gradescope, and that's that!




## Why We Use Hidden Tests

For some questions, we will have hidden tests. Considering various inputs and edge cases is a key feature of engineering AI systems, therefore you are encouraged to create your own test cases - either by running inputs by hand or adding on to our provided test suites.

While yes, it is true that you can keep "brute-force" resubmitting the autograder in a trial and error format until you get it right, we would highly recommend taking a step back and going over the high level implementation of your code and algorithms as well as carefully reading the programming document before resubmitting.

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF